



软件文化概论

南开大学·软件学院

姜桂飞

G.Jiang@nankai.edu.cn



第三讲 软件工程初步

主要内容

- 1 软件定义与特性.....
- 2 软件危机.....
- 3 软件工程定义.....
- 4 软件生存周期.....
- 5 软件工程方法.....
- 6 软件工程知识体系.....

软件的发展历程

第一阶段 电子管时代（1945-1956）

- 机器语言，汇编语言，只有少数专业人员能够编写程序，主要用于科学研究计算

第二阶段 晶体管时代（1959-1963）

- 产生操作系统，高级编程语言 FORTRAN ， COMBOL
- 程序规模小，没有形成系统化的方法，对软件的开发过程更没有进行任何管理。

软件的发展历程

第三阶段 集成电路时代（1964-1971）

- 操作系统、数据库技术及数据库管理系统、系统软件
- 结构化程序设计理念

第四阶段 （超）大规模集成电路计算机 (1971至今)

- 结构化程序设计技术（Pascal、Modula-2、Basic）
 - 1973年，C语言诞生
 - 强大的桌面系统
 - 面向对象技术
 - 并行计算
- 随着计算机应用领域的进一步扩大，软件的规模和复杂性也在不断增加
 - 在软件开发领域的地平线上出现了一朵乌云

软件无处不在



什么是软件？

“软件就是程序，软件开发就是编程序”？

软件的定义

- **软件**：计算机系统中与硬件相互依存的另一部分，它是包括程序，数据及其相关文档的完整集合。
 - **程序**是按事先设计的功能和性能要求执行的指令序列。
 - **数据**是使程序能正常操纵信息的数据结构。
 - **文档**是与程序开发，维护和使用有关的图文材料。

软件 = 程序 + 数据 + 文档

以订票软件为例

 车票

 常用查询

 订餐

① 单程

② 往返

③ 接续换乘

④ 退改签

出发地

天津



到达地

北京



出发日期

2020-09-25



学生

☐

高铁/动车

☐

查 询

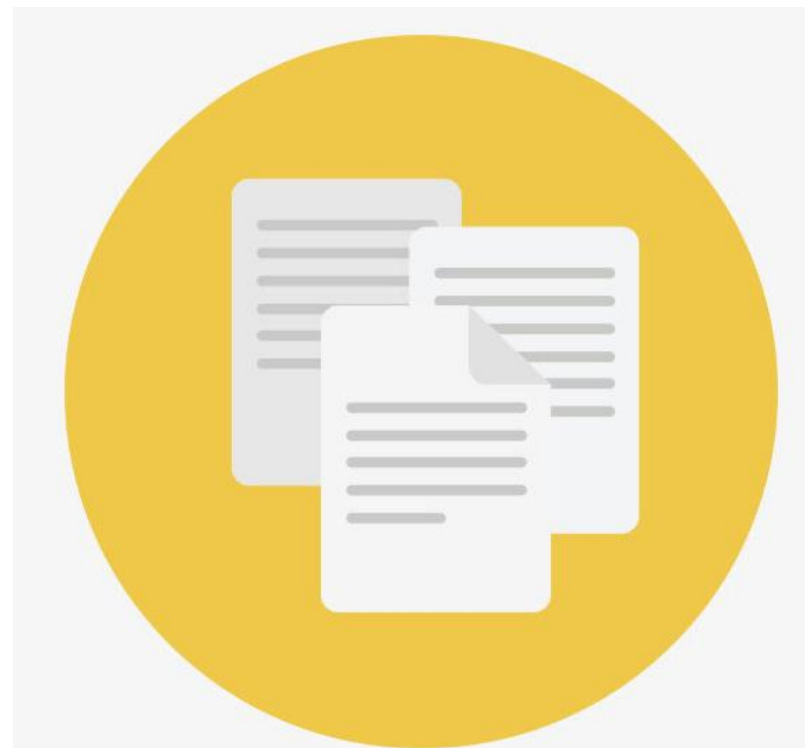
程序是完成订票操作的执行功能步骤，即执行程序

以订票软件为例

车次	出发站 到达站	出发时间 到达时间	历时	商务座 特等座	一等座	二等座 二等包座	高级 软卧	软卧 一等卧
C2608	始天津西 终北京南	05:50 06:30	00:40 当日到达	5	14	有	--	--
D718	过天津西 终北京	05:53 07:07	01:14 当日到达	--	--	有	--	有
C2002	始天津 终北京南	06:05 06:35	00:30 当日到达	5	有	有	--	--
C2610	始天津西 终北京南	06:09 06:41	00:32 当日到达	5	18	有	--	--
C2202	始天津 终北京南	06:16 06:53	00:37 当日到达	5	18	有	--	--
C2614	始天津西 终北京南	06:31 07:03	00:32 当日到达	5	17	有	--	--
C2004	始天津 终北京南	06:38 07:08	00:30 当日到达	5	20	有	--	--
C2204	始天津 终北京南	06:43 07:20	00:37 当日到达	5	14	有	--	--
C2616	始天津西	06:52	00:43	0	有	有		

数据是订票软件中需要的车次，时间，订票信息等数据

以订票软件为例



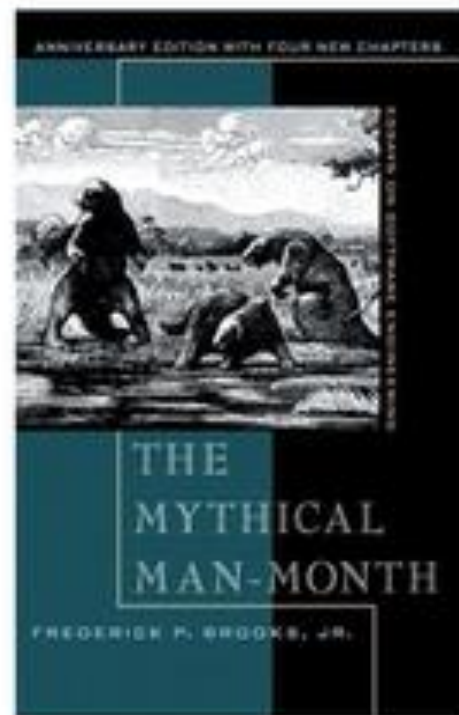
文档是指订票软件开发过程中的设计文档以及最终的用户手册等

软件特性

人月神话



弗雷德里克·布鲁克斯
Frederick P. Brooks, Jr.



Fred Brooks是北卡罗莱纳大学的计算机科学教授， 曾担任IBM OS360系统的项目经理，在计算机体系结构、操作系统和软件工程方面做出了里程碑式的 贡献，于1999年荣获计算机领域最具声誉的图灵奖。

No Silver Bullet —Essence and Accident in Software Engineering

Frederick P. Brooks, Jr.
University of North Carolina at Chapel Hill

There is no single development, in either technology or management technique, which by itself promises even one order-of-magnitude improvement within a decade in productivity, in reliability, in simplicity.

Abstract¹

All software construction involves essential tasks, the fashioning of the complex conceptual structures that compose the abstract software entity, and accidental tasks, the representation of these abstract entities in programming languages and the mapping of these onto machine languages within space and speed constraints. Most of the big past gains in software productivity have come from removing artificial barriers that have made the accidental tasks inordinately hard, such as severe hardware constraints, awkward programming languages, lack of machine time. How much of what software engineers now do is still devoted to the accidental, as opposed to the essential? Unless it is more than 9/10 of all effort, shrinking all the accidental activities to zero time will not give an order of magnitude improvement.

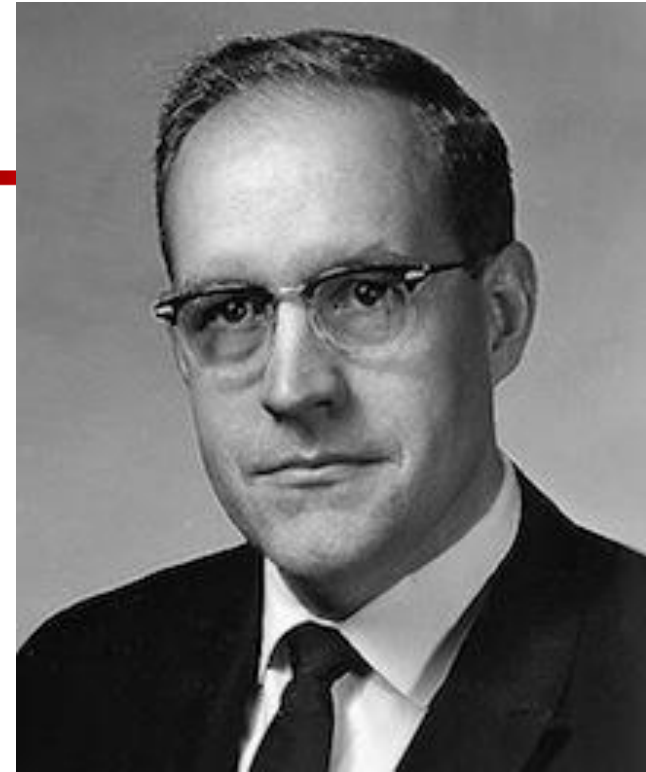
Therefore it appears that the time has come to address the essential parts of the software task, those concerned with fashioning abstract conceptual structures of great complexity. I suggest:

- Exploiting the mass market to avoid constructing what can be bought.
- Using rapid prototyping as part of a planned iteration in establishing software requirements.
- Growing software organically, adding more and more function to systems as they are run, used, and tested.
- Identifying and developing the great conceptual designers of the rising generation.

Introduction

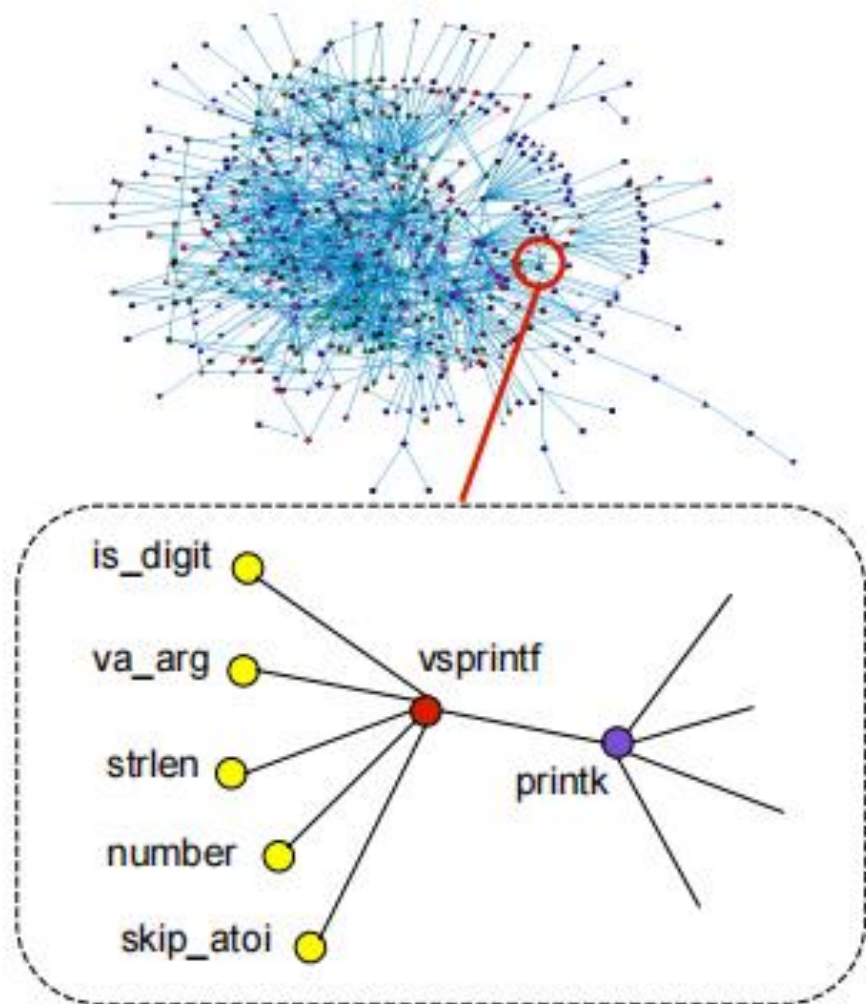
Of all the monsters who fill the nightmares of our folklore, none terrify more than werewolves, because they transform unexpectedly from the familiar into horrors. For these, we seek bullets of silver than can magically lay them to rest.

¹ Reproduced from: Frederick P. Brooks, *The Mythical Man-Month, Anniversary edition with 4 new chapters*, Addison-Wesley (1995), itself reprinted from the *Proceedings of the IFIP Tenth World Computing Conference*, H.-J. Kugler, ed., Elsevier Science B.V., Amsterdam, NL (1986) pp. 1069-76.



软件具有复杂性（complexity）、不可见性 (invisibility)、一致性 (conformity)、可变性 (changeability)等固有的内在特性。

软件特性： 复杂性



- 可以说是人类创造的最复杂的系统类型，软件的各个模块之间有各种显性或隐性的依赖关系，随着系统的成长和模块的增多，这些关系的数量往往以几何级数的速度增长
- Linux内核有630个函数，存在1814个函数调用。
- 函数在动态执行过程中呈现出更为复杂的状态。

注：图中将函数表示为节点，调用关系表示为边。

软件特性：一致性

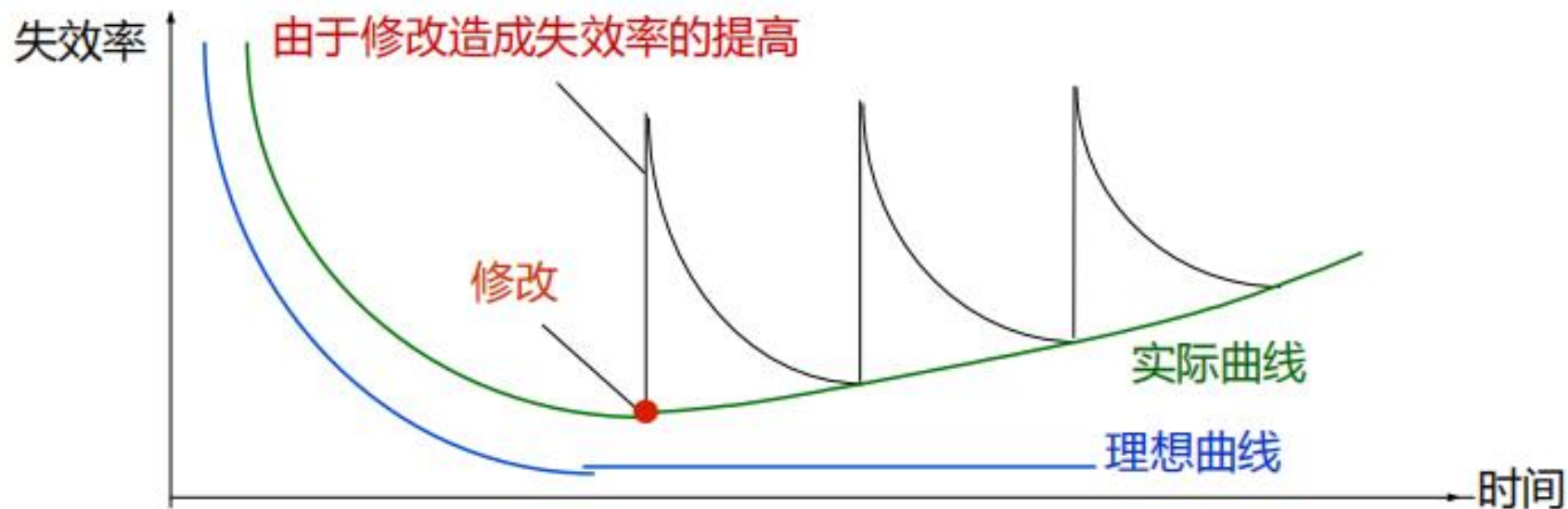
- 软件不能独立存在，需要依附于一定的环境（如硬件、网络以及其他软件）
- 软件必须遵从人为的惯例并适应已有的技术和系统
- 软件需要随接口不同而改变，随时间推移而变化，而这些变化是不同人设计的结果

软件特性：可变性



软件特性：可变性

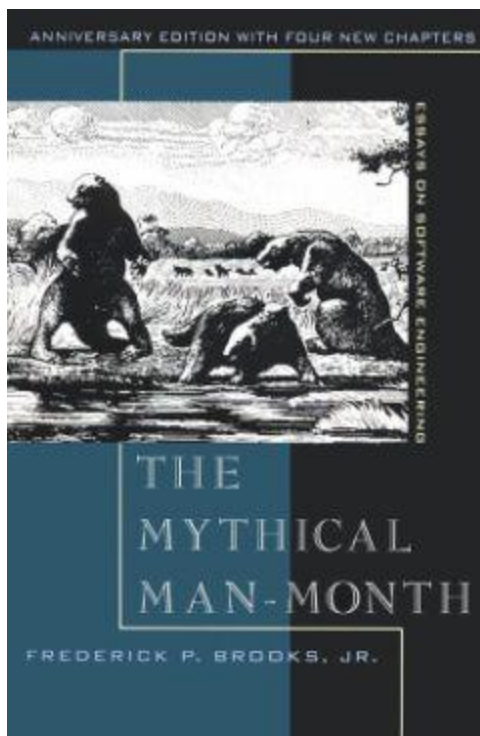
- 人们总是认为软件是容易修改的，但忽视了修改所带来的副作用
- 不断的修改最终导致软件的退化，从而结束其生命周期



软件特性：不可见性

- 软件是一种“看不见、摸不着”的逻辑实体，不具有空间的形体特征
- 开发人员可以直接看到程序代码，但是源代码并不是软件本身
- 软件是以机器代码的形式运行，但是开发人员无法看到源代码是如何执行的

软件特性：不可见性



软件人员太像“皇帝的新衣”故事中的裁缝了。当我来检查软件开发工作时，所得到的回答好象对我说：我们正忙于编织这件带有魔法的织物。只要等一会儿，你就会看到这件织物是及其美丽的。但是我什么也看不到，什么也摸不到，也说不出任何一个有关的数字，没有任何办法得到一些信息说明事情确实进行得非常顺利，而且我已经知道许多人最终已经编织了一大堆昂贵的废物而离去，还有不少人最终什么也没有做出来。

软件特性

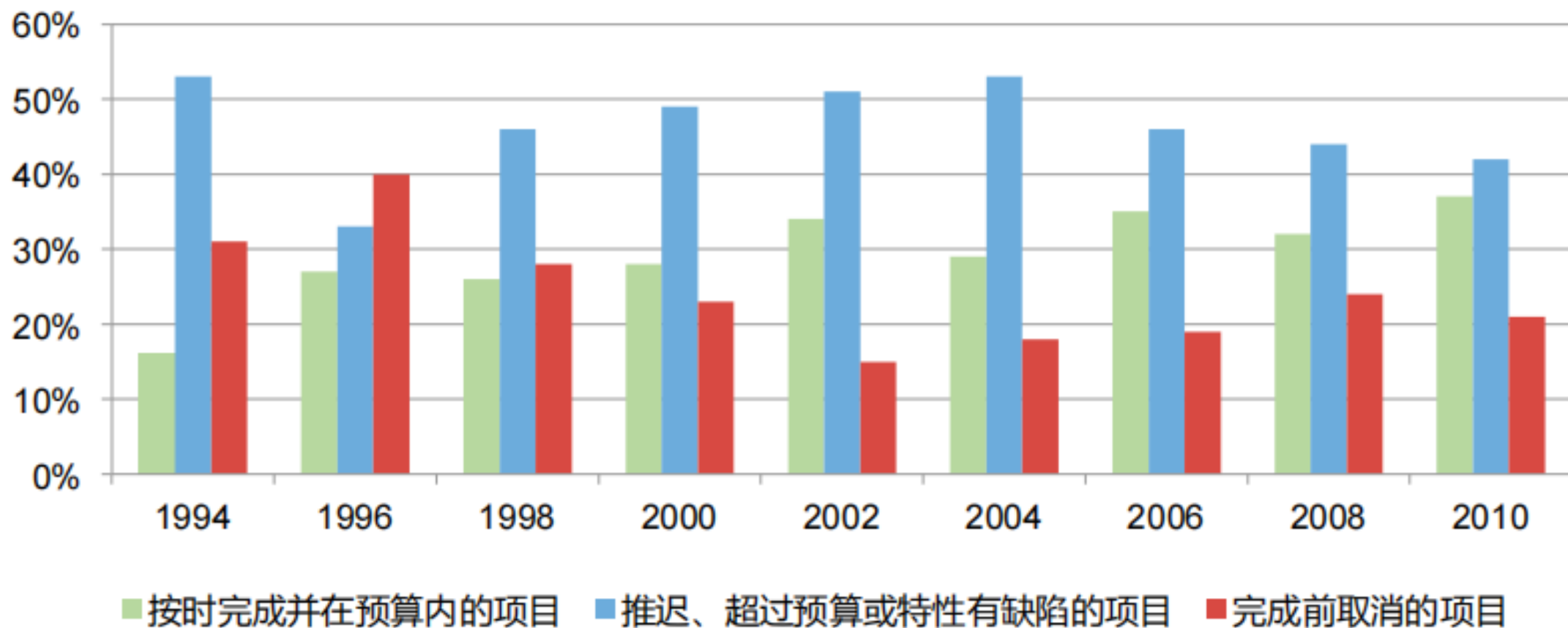
软件所具有的复杂性、一致性、可变性、不可见性等特性，使得软件开发过程变得难以控制，开发团队如同在焦油坑中挣扎的巨兽。



软件特性

- 软件是设计开发的，而不是生产制造的
 - 软件在使用过程中没有磨损、老化的问题
- 软件是定制开发的
 - 虽然整个工业向着基于构件的构造模式发展
 - 然而大多数软件仍是根据实际的顾客需求定制的
- 软件易于复制

美国Standish集团的调查报告



实例1. 欧洲ARIANE 5 火箭

- 欧洲ARIANE 5 火箭

- 1996 年6 月，耗资70 亿美元，

- 发射37 秒后爆炸

- 发射失败的原因在于软件的错误

- 软件错误

- 程序中试图将64 位浮点数转换成16 位

- 整数时产生溢出

- 缺少错误处理程序对数据溢出进行管理

- 备份软件复制而成

- 严格地遵守软件确认过程可以避免这种错误



实例2.爱国者导弹



- 爱国者导弹（Patriot missile）
 - 曾在海湾战争期间对抗伊拉克飞毛腿导弹
 - 1991 年2 月，一次对抗失利中28 名美国士兵丧生
 - 问题的症结在于导弹软件包含一个累加计时误差
- 软件错误
 - $\text{target} = f(\text{velocity}, \text{time})$
 - 计时采用系统时钟（即1/10 秒）并使用整数表达
 - $(1/10)_2 = 0.0001100110011001100110011001100\dots$ ，24 位寄存器存储导致误差 $(0.0000000095)_{10}$
 - $0.0000000095 \times 100 \text{ hours} \times 60 \times 60 \times 10 = 0.34 \text{ seconds}$

实例3. Windows Vista系统



Windows Vista™

- 该系统从2001年开始研发，整个过程历时5年半，先后有9000位开发人员投入其中，耗资60亿美元，代码规模超过5000万行。
- 按照微软公司最初的计划，该系统面世时间应该在2003年，之后推迟到2004年下半年再到2005年初，最终在取消一些高级功能。后于2006年11月正式发布。
- 由于整个系统过于庞杂，其开发管理相当混乱，以致于很多时间用在互相沟通和重新决定上。
- 从Vista Beta 1进入公开测试以来，程序错误总数已经超过2万个，这其中还不包括微软内部未公开的一些错误

实例4. 12306 网络购票系统

- 12306网络购票系统历时两年研发成功，耗资3亿元人民币，于2011年6月12日投入运行。
- 2012年1月8日春运启动，9日网站点击量超过14亿次，出现网站崩溃、登录缓慢、无法支付、扣钱不出票等严重问题。
- 2012年9月20日，由于正处中秋和“十一”黄金周，网站日点击量达到14.9亿次，发售客票超过当年春运最高值，出现网络拥堵、重复排队等现象。

软件危机（20世纪60年代末）

软件危机

- ◇ *In 1968, at a conference of "NATO".*
- ◇ *Resulting from software crisis.*
 - *Software become larger and more complex.*
 - *Development schedule is always delayed.*
 - *The software costs much more than predicted.*
 - *The software was unreliable, was difficult to maintain.*



软件危机典型表现



*Software systems are like cathedrals;
first we build them and then we pray.*

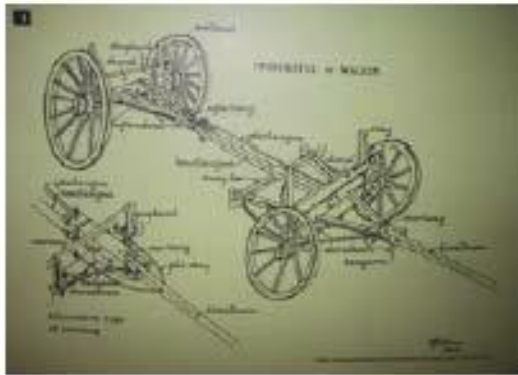
软件工程诞生

1968年，北大西洋公约组织（NATO）召开国际会议，
提出“**软件工程**”概念和术语。



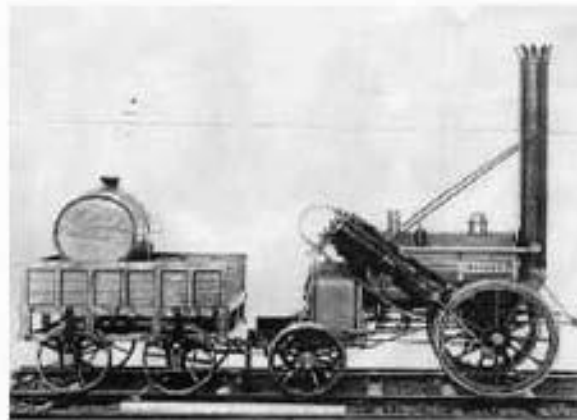
工程的含义

- 工程是将理论和所学的知识应用于实践的科学，以便经济有效地解决实际问题。



- craft
 - personal skill
 - experience
 - flexible materials

- engineering
 - tables and tools
 - recorded knowledge
 - controlled materials



- 大规模的设计与建造
- 复杂问题与目标分解
- 团队协作与过程控制

工程的含义

- 手工（**Craft**）：小规模的设计与建造
 - 简单问题与单一目标
 - 一个人控制与个人技能
- 工程（**Engineering**）：大规模的设计与建造
 - 复杂问题与目标分解
 - 多人参与，需要考虑运营、管理、成本、质量控制、安全等

什么是软件工程

- [Bauer, 1972] 软件工程是为了经济地获得能够在实际机器上高效运行的可靠软件而建立和使用的一系列好的工程化原则。



【德】Fritz Bauer

- [IEEE, 1993] 软件工程是①将系统性的、规范化的、可定量的方法应用于软件的开发、运行和维护，即工程化应用到软件上；②对①中所述方法的研究。

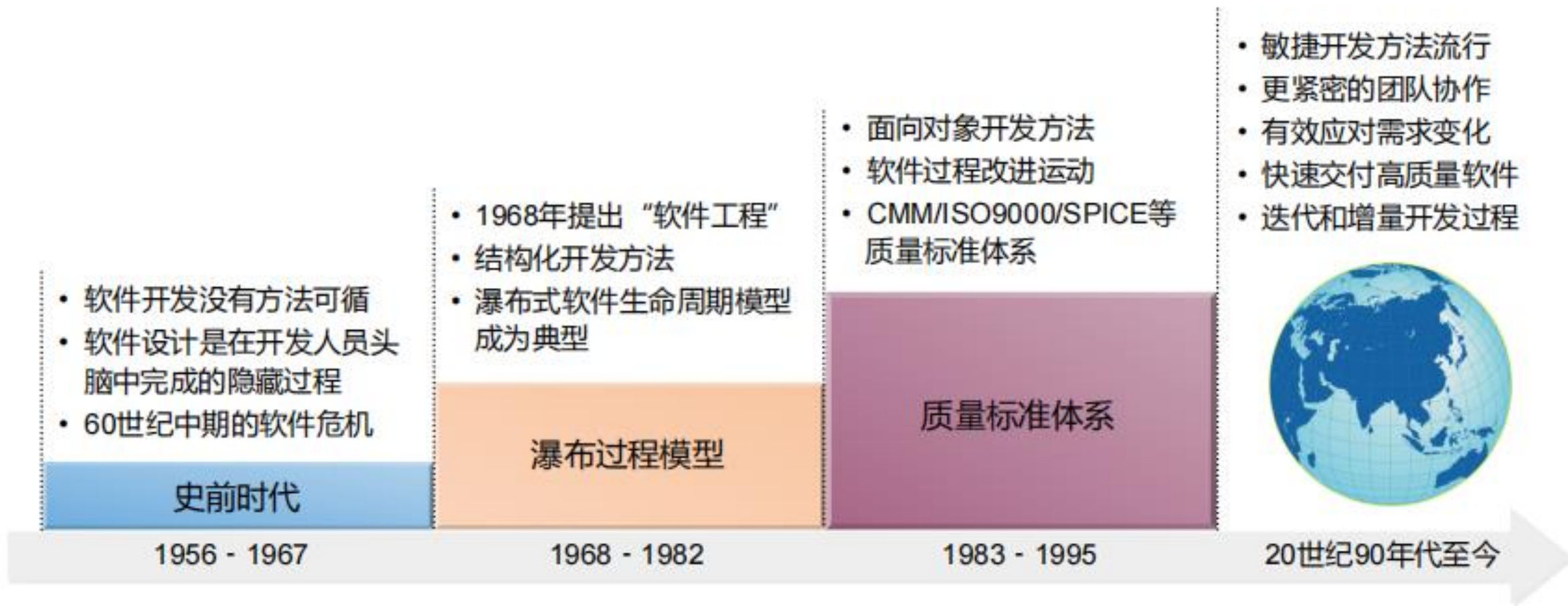
什么是软件工程

2006年中国国家标准中给出的定义

应用计算机科学理论和技术以及工程管理原则和方法，按预算和进度，实现满足用户要求的软件产品的定义、开发、发布和维护的工程或以此为研究对象的学科。

- 较低的开发成本
- 按时完成开发任务并及时交付
- 实现客户要求的功能
- 具有良好性能、可靠性、可扩展性、可移植性等
- 软件维护费用低

软件工程发展



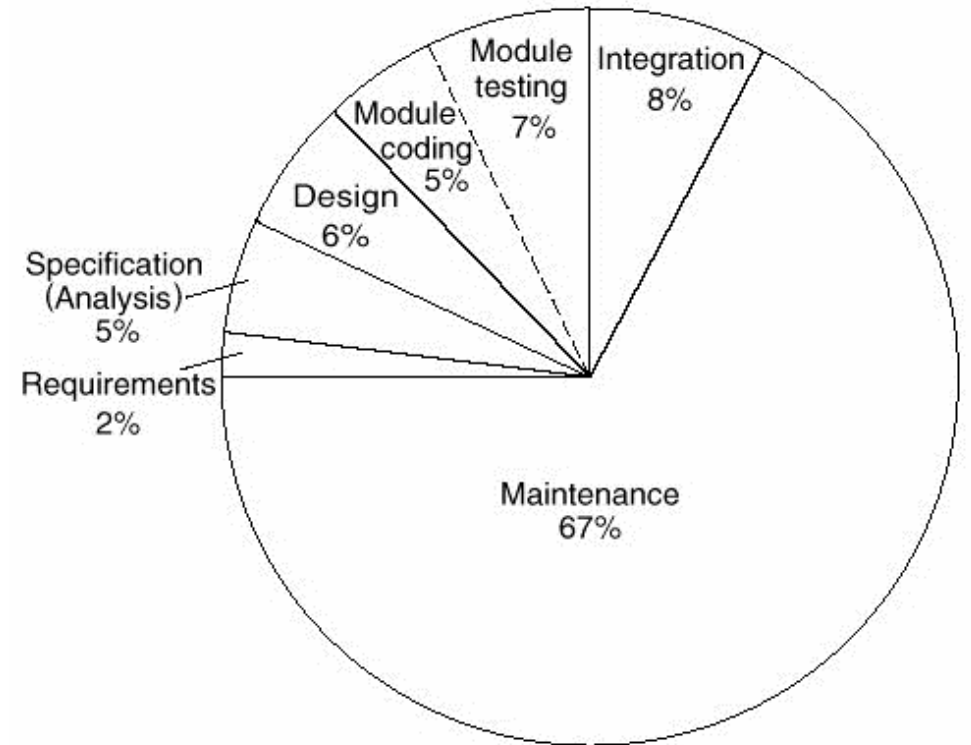
软件工程关注的焦点

- 软件质量（Software Quality）

- 软件产品与明确的和隐含的需求相一致的程度
- 通常采用一系列质量特性来描述

- 软件成本（Software Cost）

- 软件开发成本是指软件开发过程中所花费的费用
- 软件维护成本是指软件投入运行后软件变更所需的费用



来自[Elshoff, 1976], [Daly, 1977], [Zelkowitz, shaw and Gannon, 1979]和[Boehm, 1981]的统计数据

软件质量

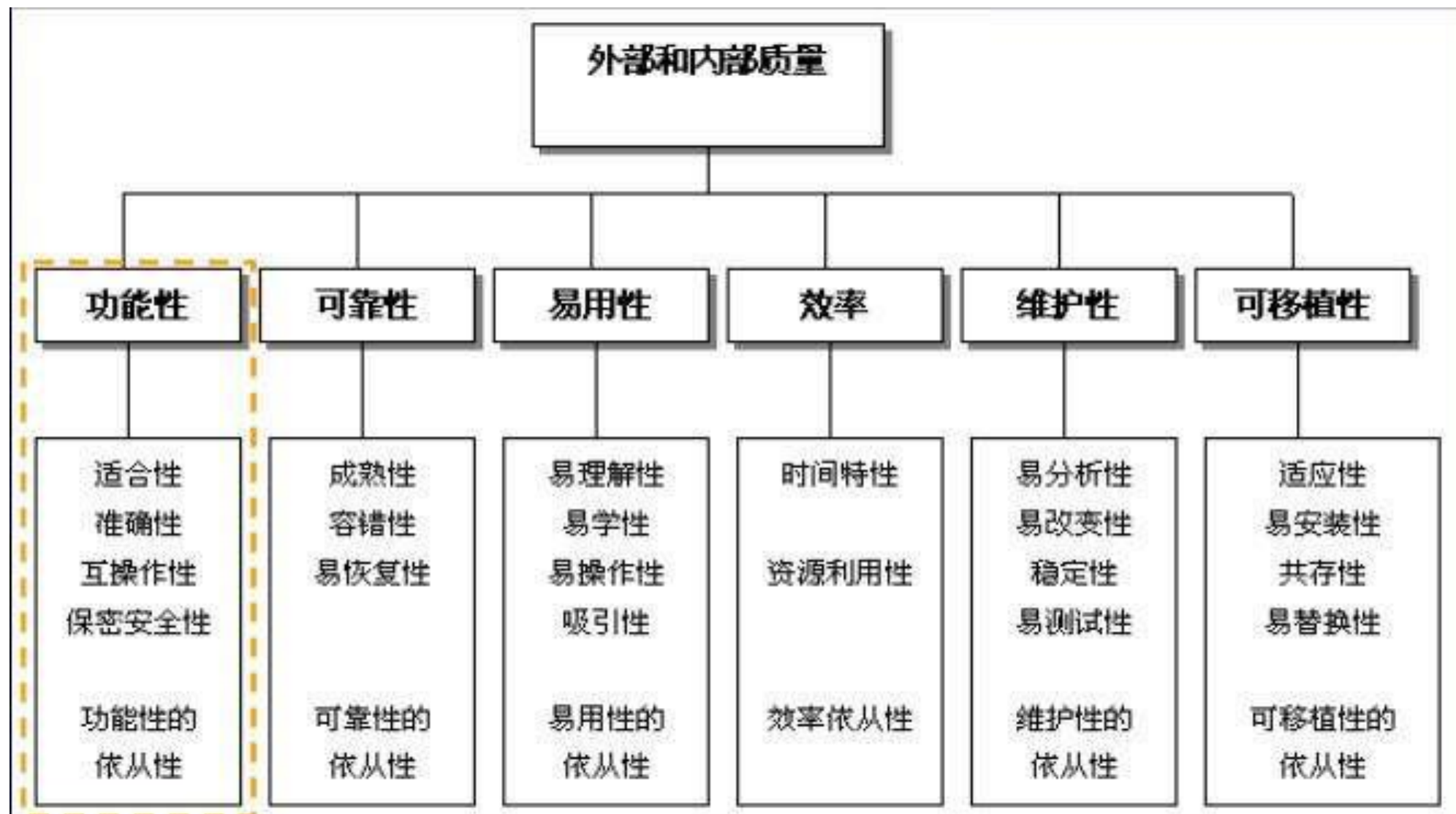
- 你同意以下说法吗？为什么？

“运行正确的软件就是高质量的软件。”

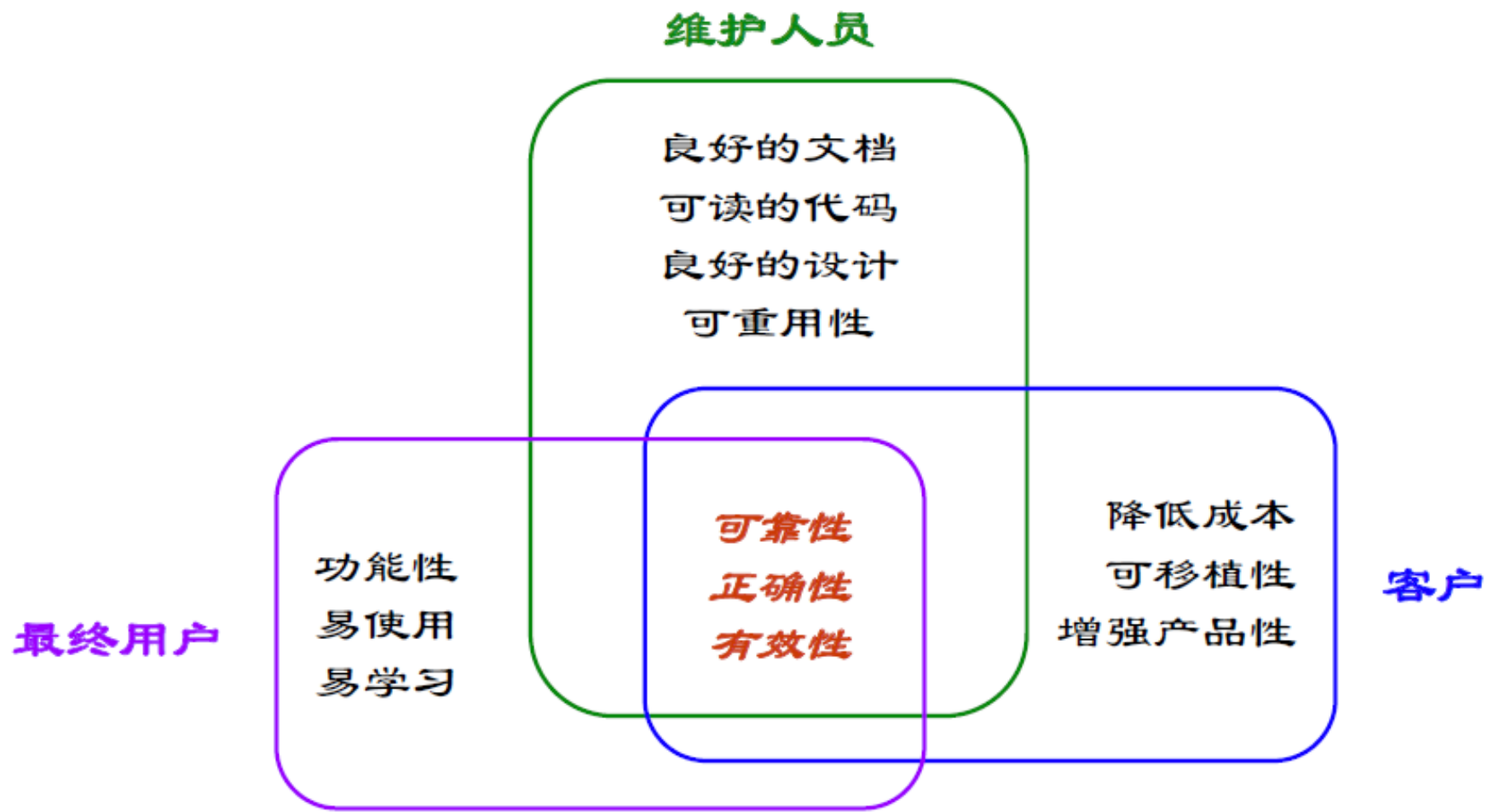
软件质量

- 你同意以下说法吗？为什么？
“运行正确的软件就是高质量的软件。”
- 软件除了提供用户所需的功能以外，还应该具有一系列反映质量的属性
 - 可维护性：软件必须能够不断进化以满足客户的需求变化
 - 可依赖性：软件必须是可靠的、保密的、安全的
 - 有效性：软件不应该浪费内存和处理器等系统资源
 - 易用性：软件必须是可用的，用户可以很方便地使用

ISO9126 质量模型

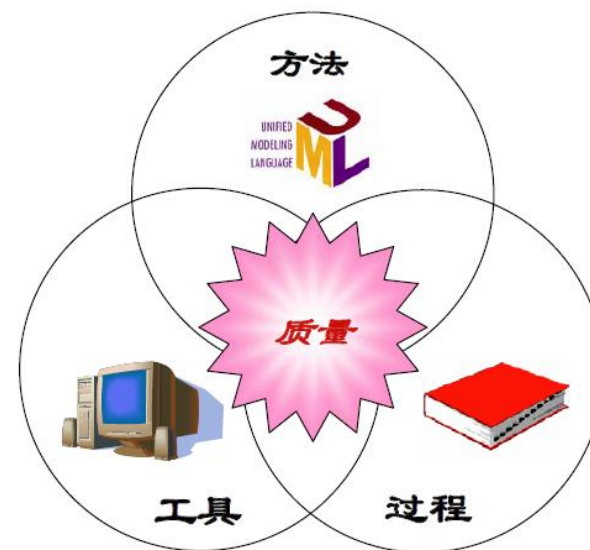


软件质量

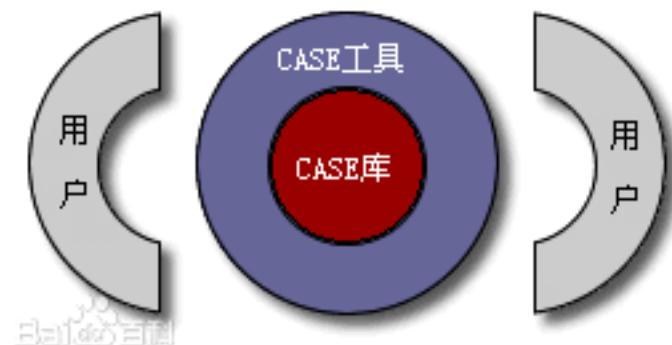


软件工程的三要素

- 软件工程以关注**软件质量**为目标，包括过程、方法和工具三个要素。
- 过程
 - 支持软件生命周期的所有活动
- 方法
 - 为软件开发过程提供“如何做”的技术
- 工具
 - 为软件开发方法提供自动的或半自动的软件支撑环境，如**CASE工具**



CASE:



软件

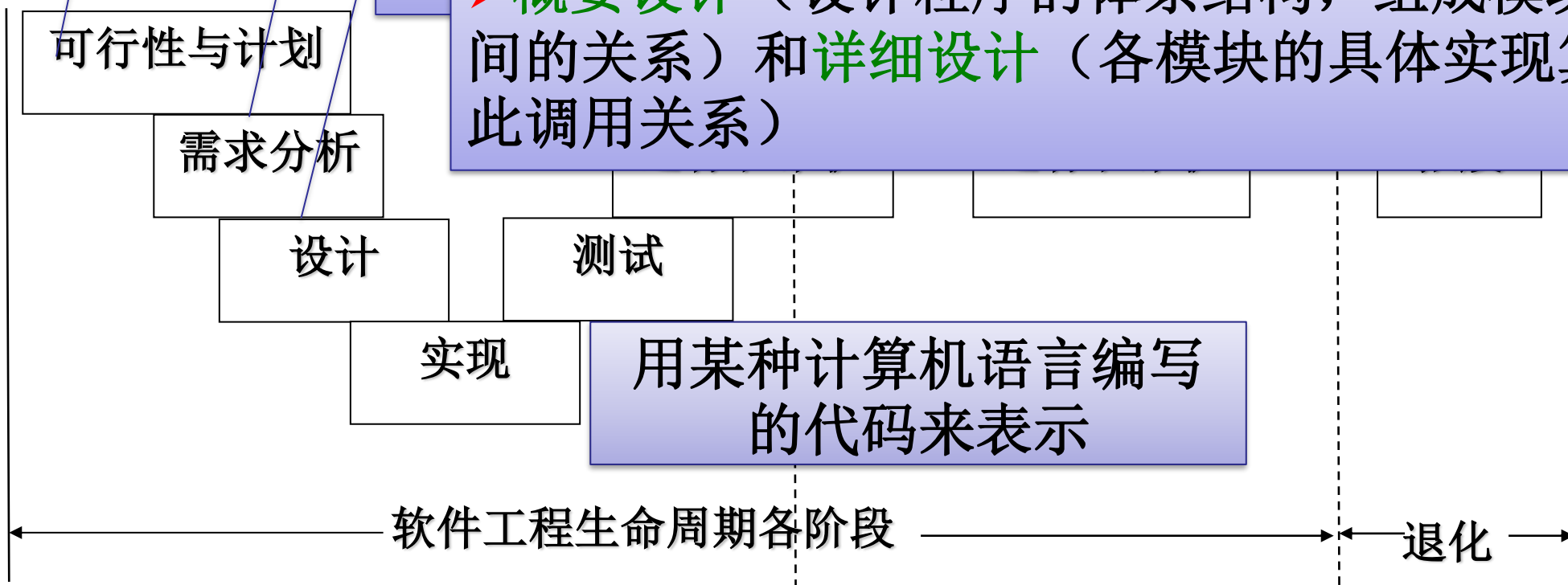
➤ 主要回答两个问题：要解决的问题是什么；这个问题具不~~有~~有~~结~~？具不~~能~~能~~结~~？

- 可行性、主要回答：用户提出的软件必须完成什么任务？
- 主要回答：软件系统如何完成，以

➤主要回答：用户提出的软件必须完成

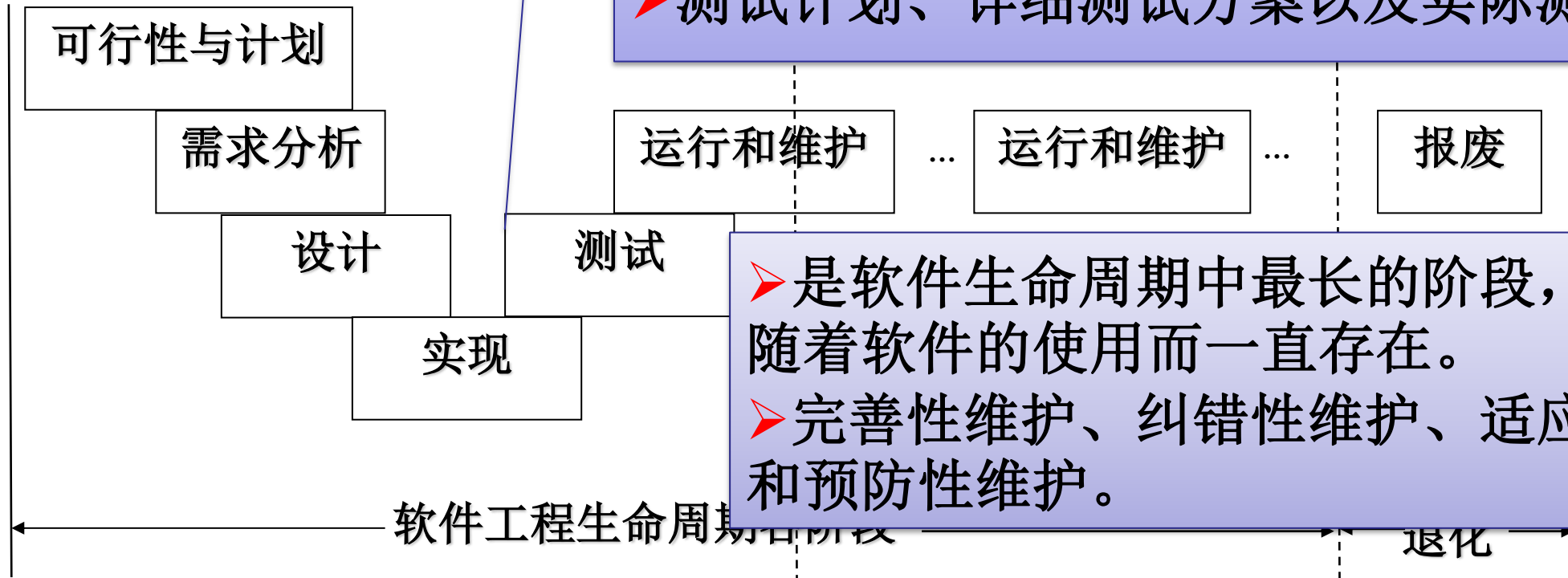
➤主要回答：软件系统如何完成，以体现用户需求？

➤ **概要设计**（设计程序的体系结构，组成模块及模块间的关系）和**详细设计**（各模块的具体实现算法和彼此调用关系）



软件生命周期

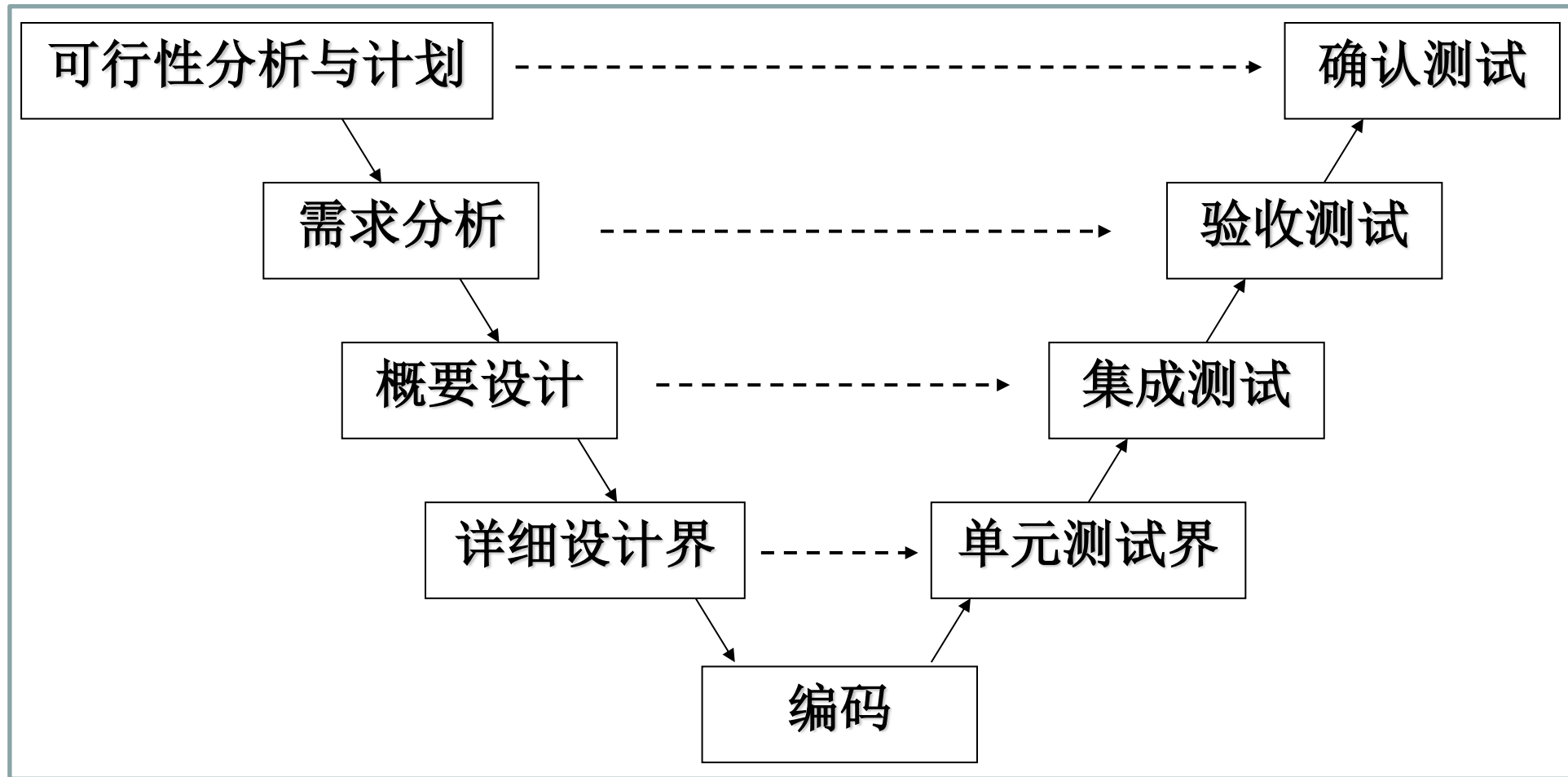
- 目的：发现程序中存在的错误，并在调试过程中修改这些错误
- 软件测试包括单元测试（实现阶段）、集成测试（设计阶段）和系统测试（需求阶段）
- 测试计划、详细测试方案以及实际测试结果等



- 是软件生命周期中最长的阶段，它将伴随着软件的使用而一直存在。
- 完善性维护、纠错性维护、适应性维护和预防性维护。

退化

软件工程过程活动的V模型



软件工程方法

- 软件工程方法是在实践中形成和发展的，在一定程度上收到程序设计方法的影响

软件工程方法

- **结构化方法**

将软件生命周期的全过程依次划分为若干个阶段，采用结构化技术来完成每个阶段的任务。

特点：

- (1) 强调自顶向下顺序地完成软件开发的各阶段任务;
- (2) 结构化方法要么面向行为，要么面向数据，缺乏使两者有机结合的机制。

软件工程方法

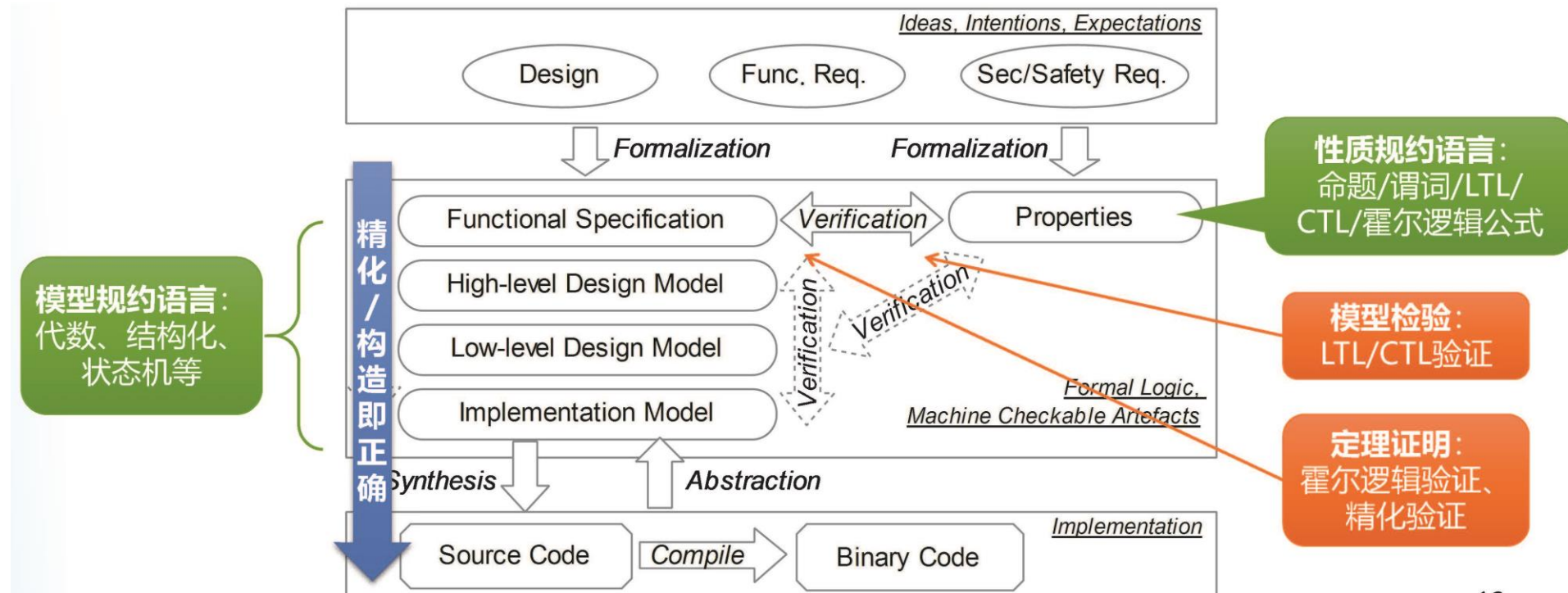
- 面向对象方法学

- 将数据和对数据的操作紧密地结合起来的方法。
- 软件开发过程是多次反复迭代的演化过程。
- 面向对象方法在概念和表示方法上的一致性，保证了各项开发活动之间的平滑过渡。
- 对于大型、复杂及交互性比较强的系统，使用面向对象方法学更有优势。

软件工程方法

● 形式化方法

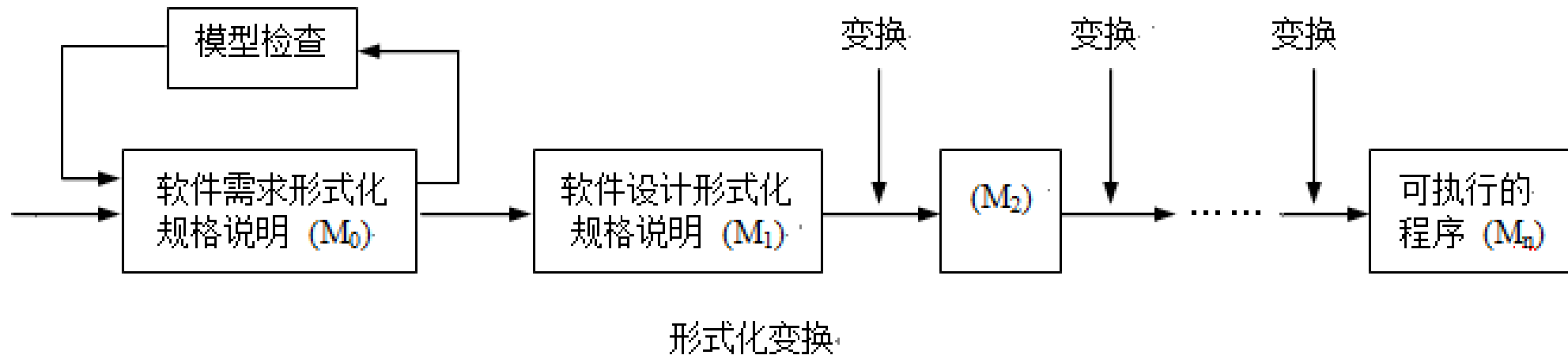
- 一种基于形式化数学变换的软件开发方法，它可将系统的规格说明转换为可执行的程序。



软件工程方法

● 形式化方法的主要特点

- 软件需求规格说明被细化为用数学记号表达的详细的形式化规格说明；
- 设计、实现和单元测试等开发过程由一个变换开发过程代替。通过一系列变换将形式的规格说明细化成为程序。



软件工程工具

需求开发

软件设计

软件构造

软件测试

软件维护

开发管理

- 软件建模工具
- 数据库设计工具

- 程序编辑器
- 程序编译器
- 程序解释器
- 程序调试器
- 集成开发环境

- 单元测试工具
- 静态分析工具
- 自动化测试工具
- 性能测试工具
- 缺陷跟踪工具

- 代码重构工具
- 逆向工程工具

- 需求管理工具
- 项目管理工具
- 配置管理工具
- 测试管理工具



软件开发的策略

- 软件复用
 - 构造一个新的系统不必从零做起，直接复用已有的构件进行组装
 - 构件是经过反复使用验证的，由其组成的新系统具有较高的质量
- 分而治之
 - 将一个复杂的问题分解成若干个简单的问题，然后逐个解决
 - 来源于人们生活与工作的经验，完全适合于技术领域
- 逐步演进
 - 软件开发是自底向上逐步有序的生长过程
- 优化折中
 - 优化：优化软件的各个质量特性，如运行速度、资源利用、用户体验
 - 折中：通过协调各个质量特性，实现整体质量的最优

软件工程知识体系

- 软件工程已从计算机科学与技术中脱离出来，逐渐形成了一门独立的学科。对其知识体系的研究从20世纪90年代初就开始了。
- **标志：**美国Embry-Riddle航空大学计算与数学系Thomas B.Hilburn教授的“软件工程知识体系指南”（Guide to Software Engineering Body of Knowledge, SWEBOK）研究项目。

软件工程知识体系

2001 年5 月完成，最新发布“SWEBOK 指南V3（2013）”

- SWEBOK 的组成

将软件工程知识分解成若干知识域，形成层次化的组成结构，15个知识域，这些知识域又划分为三类：

- **软件工程基础类**：数学基础、计算基础、工程基础、软件工程经济学。
- **软件生存期过程类**：软件工程模型和方法、软件需求、软件设计、软件构造、软件测试、软件维护。
- **软件工程管理类**：软件工程过程、软件工程管理、软件配置管理、软件质量、软件工程专业实践。

SWEBOK

- Software requirements, Software design, Software construction
- Software testing, Software maintenance
- Software configuration management
- Software engineering management
- Software engineering process
- Software engineering models and methods
- Software quality
- Software engineering professional practice
- Software engineering economics
- Computing foundations
- Mathematical foundations
- Engineering foundations

二个实践知识域

二个教育基础知识域

软件工程职业道德规范

- 软件工程不仅是技术的应用，还包括许多责任。
 - 软件工程师应当遵循本行业的职业道德规范，否则无法在这个行业中长久立足。
- 分析下面的一些行为
 - 将公司的软件代码随意复制给其他人
 - 为了个人的利益作出不切实际的承诺
 - 使用盗版软件
 - 迫于时间压力交付缺乏严格测试的代码
 - 未经允许就在别人的机器上玩游戏和上网
 -

IEEE / ACM 职业道德准则

1. 公众

- 软件工程人员应始终与公众利益保持一致。

2. 客户和雇主

- 在与公众利益保持一致的原则下，软件工程人员应满足客户和雇主的最大利益。

3. 产品

- 软件工程人员应当确保他们的产品及其改进符合尽可能高的专业标准。

4. 判断

- 软件工程人员应当具备公正和独立的职业判断力。

IEEE / ACM 职业道德准则

5.管理

- 软件工程管理者和领导者应拥护和倡导合乎道德的有关软件开发和维护的管理方法。

6.职业

- 在与公众利益一致的原则下，软件工程人员应当提高职业的信誉。

7.同行

- 软件工程人员对其同行应持平等和支持的态度。

8.自我

- 软件工程人员应当终身学习专业知识，促进合乎道德的职业实践方法。

有影响的软件工程期刊

- *Transactions on Software Engineering* (IEEE)
- *Software* (IEEE)
- *Software Engineering Notes* (ACM Special Interest Group)
- *Transactions on Software Engineering and Methodology* (ACM)
- *The Journal of Systems and Software* (Elsevier)
- *Proceedings of the International Conference on Software Engineering* (ACM / IEEE)
- *Proceedings of the International Conference on Software Maintenance* (IEEE)
- *Software Maintenance: Research and Practice* (Wiley)

总结

- 重点把握
 - SE的基本概念
 - 软件质量
 - 软件生存周期
- 了解
 - 结构化方法与面向对象方法
 - SE知识体系
 - SE职业道德

参考文献

- 《计算机科学概论》（人民邮电出版社）第7章（中/英）
- 《计算思维导论》6.5节